

NetraSetu — Technical Documentation

PS 26038 — Explainable AI for Diabetic Retinopathy Screening in Rural India Team "Game Of Codes": Tanuj (team lead; ML layer, mobile app, integration), Saad (backend, database), Krrish (lead frontend developer), Parth (frontend, presentations), Vedant (frontend, presentations), Kankshi (backend and ML layer support)

This document explains what the system is, how its parts fit together, and *why* it's built the way it is — every component, every design decision, and the current state of each part. Every ML number here matches `docs/ML_BENCHMARKS.md`, which carries the full populations, intervals and caveats.

1. The problem, and why the architecture looks the way it does

India has 20,944 ophthalmologists, about 15 per million people (AIIMS Delhi national survey, 2025). Blindness is $1.37\times$ more prevalent in rural areas, and 90% of DR-related vision loss is preventable with timely referral.

The real task isn't "build an AI that detects DR." It is: **let a non-specialist technician at a rural PHC do what only a specialist could do before, and let that specialist trust and act on the AI's output in seconds.** Every architectural choice below traces back to that sentence.

Edge/cloud split. Rural PHCs have unreliable internet and modest hardware. Compressing the grading models to run there would sacrifice exactly the accuracy the system is judged on. The only step that genuinely needs to run locally is the image-quality check: the technician needs a retake decision before the patient leaves the chair. So **local does quality gating only; central does all grading.** This applies identically to both front-ends.

Capture source. A phone's bare camera cannot produce a usable fundus photograph; that is an optical limitation, not a software one. Both front-ends therefore take images from a dedicated fundus camera. The mobile app imports those images from the gallery, and it also offers a dedicated fundus-lens capture mode (`mobile_lens` preset). Neither front-end calls an external or unversioned inference endpoint.

Human-in-the-loop, always. The AI never tells a patient they have a disease. Every "probably has DR" outcome is confirmed by an ophthalmologist before it becomes a message to the patient.

Two front-ends, one spec. The desktop web app and the Expo mobile app implement the same technician workflow against the same central API contract. They differ in exactly one respect: how the image gets onto the device.

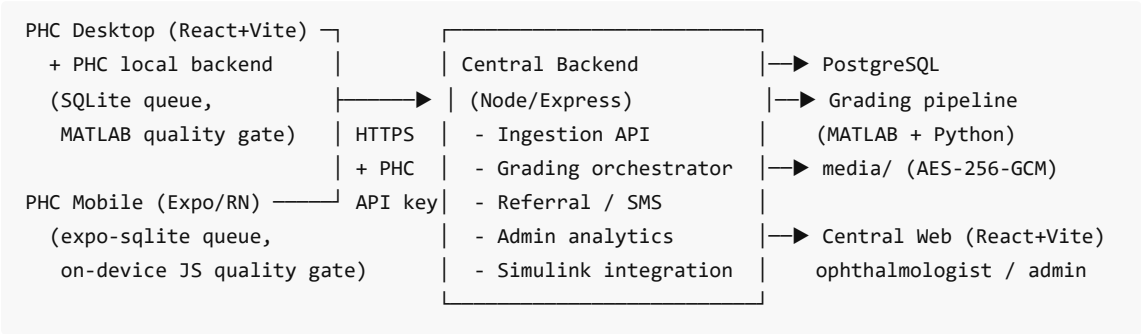
No result ever substitutes for a failure. A network failure, a timeout and a working result must be distinguishable on screen. A failed request surfaces as an error or genuinely queues for retry, never as a fabricated result. Mock data only ever appears behind an explicit, visible "DEMO DATA" banner (`DATA_MODE=mock`).

2. Actors

Actor	Touches
PHC technician	Either local front-end (desktop or mobile)
Patient	Gives verbal consent (timestamped at registration); receives an in-person result and/or SMS
Ophthalmologist	Central web app: review queue, case detail, decision controls
District health administrator	Central web app: dashboard, system health, referral tracker, resource recommendations

ASHA / community health worker	Referral follow-up, assigned via the admin interface
--------------------------------	--

3. High-level architecture



Where MATLAB lives:

- the desktop quality gate;
- camera-fingerprint calibration;
- segmentation orchestration;
- the rule engine;
- conformal and calibration math;
- explainability and the evidence report;
- the Simulink resource model.

Branch A (the CNN) and all segmentation models are trained in PyTorch, exported to ONNX and imported into MATLAB's Deep Learning Toolbox. Tensor-level parity was verified for all 9 model artifacts, with maximum differences of 1×10^{-6} to 4×10^{-5} .

Python remains on the serving path for preprocessing and the segmentation worker. The preprocessing chain is implemented exactly once, and that implementation both trains and serves the model: an earlier MATLAB reimplementaion was removed after it flipped real grades via a residual invisible on synthetic test tensors.

No Redis: the grading queue runs in-process in the central backend.

3.1 MATLAB and Simulink in this system

Component	MATLAB product(s)	Where it runs	Status
Local quality gate	Image Processing Toolbox; MATLAB Compiler (standalone tier)	PHC desktop	Live , both tiers — a persistent MATLAB engine by default, and a compiled standalone executable (runs on the free MATLAB Runtime, no license needed on the PHC machine), verified byte-for-byte against it. Ships in the standalone PHC (\$4.4, \$12.3).
DR severity classifier inference	Deep Learning Toolbox	Central	Live by default, via a persistent MATLAB session serving the network imported from ONNX.

Segmentation / localization inference	Deep Learning Toolbox	Central	Live for all four models — vessel, optic-disc/fovea localization, hard-exudate and microaneurysm/haemorrhage segmentation all run through the same persistent MATLAB session (see §6.6).
Rule engine (ICDR 4-2-1)	MATLAB (base, no toolbox)	Central	Live for every graded case, including the fovea-reliability gate that skips quadrant logic when localization isn't trustworthy.
Conformal prediction / calibration math	MATLAB (base and Statistics and Machine Learning Toolbox)	Central	Live , feeding the real tier decision on every case; 90/90 automated tests pass (ML_BENCHMARKS.md §7).
Grad-CAM + lesion-attention consistency	Deep Learning Toolbox	Central	Live for both halves, on every graded case, not only in an offline script.
Camera-fingerprint calibration	Image Processing Toolbox	Central	Live — its output feeds the tier decision through the camera-validation path, a real effect on routing.
Evidence-report PDF	MATLAB Report Generator (core-MATLAB fallback renderer)	Central	Live , generated on demand for any graded case.
District resource model	Simulink + SimEvents	Central (scheduled)	Built and runnable. The weekly cross-validation of this model against an independent pure-MATLAB reference implementation is a software self-test, not the source of the daily Resource Recommendations screen — that screen's numbers come from the reference model directly, with the Simulink model validating it periodically (§7).
District resource model, standalone app	Simulink Compiler + MATLAB Compiler	Any Windows PC (free MATLAB Runtime)	Built. SimEvents blocks can't generate code, so this app runs a separate, code-generation-capable model proven to reproduce the reference model's results exactly. See §7.1.
Interactive full-pipeline dashboard, standalone app	Simulink Compiler + MATLAB Compiler	Any Windows PC (free MATLAB Runtime)	Built. The watchable whole-pipeline demo with live sliders and switches that work mid-run, packaged on a code-generation-capable twin of the SimEvents model. See §7.2.

4. Local PHC application (both front-ends)

4.1 Screens

Screen	What it does
Patient Registration	New-patient form. Fuzzy duplicate check by name/age/phone before an ID is minted. Timestamped verbal-consent checkbox. Patient symptom/risk questionnaire (diabetes duration, glycemic control, BP, pregnancy, symptoms; tap-only, no skip)
Capture	Desktop: camera trigger / import. Mobile: gallery import or fundus-lens mode. Tags left/right eye before anything else
Quality Result	Pass / borderline / fail with a specific reason (blur, dark, field of view, glare, motion, eyelash occlusion, ...). A real quality score when one exists, "not available" when it doesn't, never a fabricated number. The engine that ran is shown
Capture Metadata Questionnaire	Device, pupil status, lighting, observed issues. Tap-only, under 45 seconds, runs after the quality gate
Local Queue	Captured → Quality-Passed → Synced → Result-Pending → Result-Delivered, with a storage-pressure indicator
Sync Status	Online/offline indicator and pending count. "Synced" means central actually accepted the case (HTTP 201), never merely that a request returned

Final flow on both apps: registration + patient questionnaire → capture → quality gate → capture-metadata questionnaire → queue → sync.

4.2 Local backend

Service	Responsibility
Quality-Gate Engine	Desktop , tried in this order, each result stamped with which engine produced it: (1) the compiled standalone executable on the free MATLAB Runtime, no license needed on the PHC machine; (2) a live MATLAB session; (3) a verified JS port , only when MATLAB can't launch — checked against MATLAB with zero mismatches across dozens of test images; (4) otherwise an honest "quality check failed" result — the image is kept and no verdict is invented. Mobile : its own on-device JS port (engine <code>js-device</code>), with its own parity test against MATLAB
Local API	Wraps capture + quality gate and reads/writes the local database
Sync Manager	Prioritizes by urgency tier, then age. Sends a lightweight case-summary packet ahead of the full image on thin connectivity. Uploads full images in resumable chunks with hash verification. Every submission carries the local <code>capture_id</code> as an idempotency key, so a retry never creates a duplicate case

4.3 What differs on mobile, and only this

- **Local persistence:** a real embedded database (`expo-sqlite` , per-record), not a serialized blob.
- **Identity:** the exact same collision-safe ID scheme as desktop (PHC code + timestamp + random suffix).
- **Questionnaires:** both apply in full.
- **Login:** the mobile app authenticates against the paired PHC PC's technician accounts over an encrypted peer channel, with an honest offline-cache fallback.

4.4 Standalone PHC (downloadable, no MATLAB licence)

A complete PHC station packaged as a Windows zip (`NetraSetu-PHC-standalone.zip` , distributed outside git). It contains:

- the PHC backend and the built web app;
- a bundled Node runtime;
- the **compiled quality gate** (`qualityGate.exe`), which needs only the free MATLAB Runtime R2026a;
- `Start-PHC.cmd` , `Add-Technician.cmd` , `Check-Setup.cmd` ;
- one `settings.env` for site identity (`PHC_CODE` , `CENTRAL_API_URL` , `PHC_ID` , `PHC_API_KEY`).

Patient data is kept in `data\` , separate from the program, so replacing the program keeps the data.

It was tested end to end against a local central (2026-10-02): registration, capture, compiled gate, sync, MATLAB grading at central, and "RESULT READY" back at the station. Restarts, install paths containing spaces, offline capture and enforced login were also checked.

Not yet tested: a PC that has only the MATLAB Runtime installed and no MATLAB.

5. Central system

5.1 Ophthalmologist interface

Screen	Components
Review Queue	Sorted so the least-confident cases surface first, not the most severe. Shows patient ref, eye, PHC, capture time, both branches' grades, tier and claim status. Auto-refreshes
Case Detail	Fundus image with toggleable Grad-CAM overlay. Lesion-evidence panel: microaneurysms, haemorrhages and hard exudates measured, cotton-wool spots shown as unmeasured. Both grading branches side by side with a disagreement flag. Calibrated confidence and conformal tier. Review-duration timer. Engine-provenance details (collapsed by default). Downloadable evidence-report PDF
Decision Controls	Opening a case claims it; a second reviewer sees who holds it and can't submit a conflicting decision. Confirm, or Override with a structured reason, optional free text and a corrected grade. On branch disagreement, Confirm is not available: the reviewer must pick an explicit final grade
Case History	Per-patient, per-eye longitudinal view

5.2 District admin interface

Screen	Components
Dashboard	Cases today/week/total, average review time, override rate, average confidence, cases by PHC
PHC Health (includes System Health)	Every PHC with last sync time, 24-hour volume, pending/failed count, and active/silent status. On the same screen, the consolidated system-health view (<code>GET /api/v1/admin/system-health</code>): silent PHCs (one definition and threshold everywhere), stuck grading jobs, MATLAB session health, and referable cases unreviewed past a threshold. There is no separate System Health route; the case status banner uses the same data

Referral Tracker	referred → contacted → attended / lost. Assigned-worker field. Automatic manual-follow-up state when SMS delivery fails
Resource Recommendations	Staffing/routing guidance from the Simulink resource model

5.3 Central backend services

- Ingestion API (dedup on `capture_id`)
- Grading Pipeline Orchestrator (preprocessing → camera calibration → segmentation → dual-branch grading → confidence routing → rationale)
- Referral & Notification Service (Twilio SMS, manual follow-up on failure)
- Admin Analytics Aggregator
- Simulink Integration
- Grading Job Watchdog
- MATLAB Session Supervisor (auto-restart, health reporting)
- Auth Service
- Audit Logging
- Site & Camera Probation Tracking

5.4 Database

PostgreSQL with migration tooling (`node-pg-migrate`). Core tables:

- `patients` , `cases` ;
- `grading_results` : both grades, agreement flag, confidence, uncertainty, conformal tier, claim state;
- `segmentation_outputs` : lesion counts per category, NV score recorded for research only, fovea-reliability flag;
- `explainability_outputs` : Grad-CAM path, report path, engine provenance;
- `ophthalmologist_reviews` , `referrals` , `phc_sites` , `users` , `access_log` .

5.5 API contract

One contract, `docs/api-contracts.md` , consumed identically by both front-ends. Core endpoints:

- `POST /api/v1/cases` (idempotent on `capture_id`);
- `POST /api/v1/cases/summary` ;
- chunked upload endpoints;
- `GET /api/v1/ophthalmologist/queue` ;
- `POST /api/v1/cases/:id/review` ;
- `GET /api/v1/phc/cases/:captureRef/report` ;
- `GET /api/v1/admin/*` (dashboard, system health, PHCs, referrals);
- `POST /api/v1/auth/*` .

6. ML pipeline

6.1 Image quality assessment (local)

Classical computer vision in `quality-gate-matlab/qualityGateMain.m` and its four `assess*.m` helpers, with per-camera presets in `cameraPresets.json` (`default` , `mobile_lens`).

Seven scores:

Score	How it is computed
focus	Laplacian variance
illumination	Mean grey distance from 100
field of view, coverage	Largest filled bright region
glare	Saturated centre pixels
motion	Horizontal vs vertical gradient variance
occlusion	Dark pixels inside the retinal disc's convex hull

Decision order: insufficient FOV → glare → motion → low illumination → blur → eyelash occlusion. If none of these fires, the image is **borderline** when the mean of focus, illumination and FOV is below 0.7, otherwise **pass**.

The same gate runs in four forms:

- **MATLAB**, the reference;
- the **compiled exe**, which reproduces every MATLAB sub-score to 1e-9;
- the **desktop JS port**, rewritten 2026-10-02 as a step-by-step port (exact `rgb2gray` weights, N-1 variance, MATLAB's `imclose` border semantics, `bwconvhull` built from pixel-edge midpoints, the same presets). Against MATLAB it had 0 mismatches on 58 images, every decision identical, and a worst score gap of 3.3e-4;
- the **mobile TypeScript port**.

The thresholds are engineered heuristics and have not been validated against a gradability-labelled dataset (§10).

6.2 Preprocessing (central)

Ben Graham preprocessing only: circular retinal crop → resize → Gaussian-subtraction contrast ($4 \cdot \text{img} - 4 \cdot \text{blur} + 128$). It is implemented exactly once (`preprocessing/ben_graham.py`) and imported by both training and serving (see §3 for why).

CLAHE is **not** part of the serving chain. Adding a CLAHE stage that training never used dropped agreement with the model's own outputs from 100% to 57.7% (`inference/branchAInfer.py` header). `clahe_enhance.py` remains in the repo only as an experiment.

6.3 Camera-fingerprint calibration (central)

Identifies the capture device family from vignetting, aspect ratio and colour-channel gains, cross-checked against the technician-reported device. A mismatch, or an unvalidated camera family, feeds confidence routing as a reason for human review, not as a pixel-level correction.

6.4 Optic disc / fovea localization (central)

A U-Net heatmap regressor with 16 px (disc) / 32 px (fovea) mean error at native resolution on IDRiD. The fovea heatmap occasionally has no real peak, which would silently rotate the quadrant axis the rule engine relies on. A peak-confidence gate (threshold 0.37, raw heatmap units) marks such cases `foveaUnreliable` : the rule engine then skips quadrant-dependent logic, and the case is forced to at least Tier B.

6.5 Vessel segmentation (central)

A U-Net trained on CHASE_DB1 (Dice 0.777 held out). There is a real cross-dataset gap (Dice 0.619 on DRIVE). The live threshold is a fixed 0.5; a per-domain adaptive threshold is a natural next-round improvement.

6.6 Lesion segmentation (central)

- **Red lesions (microaneurysms + haemorrhages):** the deployed model is a 3-class U-Net (background / microaneurysm / haemorrhage) with class-specific minimum-area filters (microaneurysm 5 px, haemorrhage 10 px). It reports real, separate counts per quadrant — an earlier, simpler version could only report one combined total. Dice is 0.599 merged (microaneurysm 0.442, haemorrhage 0.571) on 16 validation images; that is early evidence, not a weak signal, and doesn't yet establish a measured accuracy gain over the earlier version (0.535) on its own.
- **Hard exudates:** Dice 0.583 per image / 0.733 global on IDRiD.
- **Cotton-wool spots:** an explicit scope exclusion (data too sparse), always reported as unmeasured, never as a false zero.
- **Neovascularization:** a suspicion score was built and tested. It **failed validation** (AUC 0.29 IDRiD, 0.38 Messidor-2, both below chance), so it influences no decision.

Standing rule: the lesion size filters and the rule engine's thresholds are one jointly calibrated unit. Changing one without recalibrating the other silently rescales what the thresholds mean.

6.7 DR severity classification — two independent branches (central)

Branch A (CNN). The deployed classifier is an EfficientNet-B0 at 512×512, with an ordinal-aware loss and class weighting. It was trained on APTOS 2019 + IDRiD, plus an EyePACS subset in the 5-class loss and photometric domain augmentation.

On the 628-image held-out test set:

Metric	Result
QWK	0.884
Referable sensitivity / specificity (argmax)	92.7% / 92.4%
Grade-4 recall	57.4% (31/54)

The live referral decision uses a calibrated threshold on $P(\text{grade} \geq 2)$, set at 0.3873, not the argmax grade. By 50-fold cross-fit ($n = 1,161$), sensitivity is 95.0% and specificity 91.0% in-domain.

Model choice. The deployed classifier was chosen over a candidate that technically scored marginally higher in training, because its auto-clear tier is far more stable on an unseen camera. This deviation is disclosed in `ML_BENCHMARKS.md §1d`.

Branch B (rule engine). Plain, testable code implementing the ICDR "4-2-1" rule on quadrant-mapped lesion counts, auditable line by line against the clinical text. Its frozen thresholds are `RED_FLOOR=3` , `GRADE3_QUAD_MIN=3` and `RULE_MAX_GRADE=3` . It is deliberately capped at grade 3, since no validated neovascularization signal exists. Exact agreement with ground truth is 60.2% on IDRiD's official test split ($n = 103$).

Fusion. Agreement supports the tier. Disagreement unconditionally forces mandatory review with an explicit resolution. The rule engine's value lies in being an independent, auditable second opinion and a disagreement tripwire, not in standalone accuracy.

6.8 Confidence routing (central)

One decision per case combines:

- temperature-scaled confidence;
- referable-stratified conformal prediction;

- branch agreement;
- camera validation / probation status;
- quality flags, including `foveaUnreliable` .

Tier	Meaning	Consequence
A — auto-clear	Singleton conformal set, high confidence, branch agreement, validated camera, no quality flag	Skips the ophthalmologist queue
B — AI-assisted review	Narrow conformal interval, or an unvalidated camera	Full rationale report; target review under 30 s
C — full manual review	Wide interval, branch disagreement, CNN grade-4, probation, or quality flag	No shortcuts

In-domain safety (cross-fit, $n = 1,161$): **0% false auto-clears** of referable and of grade ≥ 3 cases, and **zero grade-4 cases auto-cleared** across 1,000 fold assignments.

On an unseen camera (Messidor-2, $n = 872$): the referable false-auto-clear rate rises to 2.3% (upper bound 5.3%), and sensitivity at the shipped threshold is 75.2%. **This is why unvalidated cameras cannot auto-clear.**

Two coded safety clauses are structurally subsumed by the calibrated threshold today, so they are not presented as active safety layers:

- "P(grade 3) + P(grade 4) > 0.5 $\Rightarrow$ referable";
- "referable threshold demotes Tier A."

6.9 Explainability (central)

Grad-CAM on the classifier's final convolutional layer, restricted to the retinal region, with a lesion-attention consistency score that checks the heatmap against the segmentation masks. It is assembled together with the lesion evidence and the rule-engine criteria that fired into one rationale and a downloadable evidence-report PDF.

The Grad-CAM image is the model's own cropped working image, not the original capture, and the UI captions this.

6.10 Symptom + risk questionnaire — queue-ordering hint, not a confidence adjustment (central)

The questionnaire (diabetes duration, glycemic control, BP, symptoms) never adjusts the classifier's confidence or grade — the image-based result is never overridden by what the patient reports. What it does feed is a queue-ordering hint: a random-forest model trained on synthetic data helps sequence which cases a reviewer sees first, and the questionnaire answers are shown as display text on the case report for clinical context. The grade itself comes from the image alone, always.

6.11 Continual learning (central)

Ophthalmologist overrides and their structured reasons are captured in the same transaction as the review. They are exportable with consent enforcement and deduplication (`scripts/exportTrainingSet.js`). Scheduled fine-tuning, the validation gate and promotion are **not implemented this round**: corrections are captured and exportable, and retraining is a manual, future step.

7. Systems layer — Simulink resource model

A discrete-event simulation built in SimEvents.

- **Entities:** patient images.
- **Arrivals:** each PHC's acquisition rate.
- **Network:** a bandwidth-constrained queue across good / poor / very-poor connectivity tiers.
- **Review:** ophthalmologist review as a limited-capacity server (~30 s for Tier B, several minutes for Tier C, with Tier C pre-empting Tier B).
- **Outputs:** queue length over time, average wait and bottleneck location.

What actually feeds the Resource Recommendations screen. The screen's daily numbers come from a lightweight pure-MATLAB queueing model that runs fast enough for an on-demand refresh. The full SimEvents model is real and runs to completion on its own weekly schedule, and serves as an independent cross-check against that lighter model's output — a second, differently-built implementation of the same queueing logic agreeing with the first, not a duplicate of it feeding the screen directly.

Bandwidth and timing parameters are modeled assumptions, not measured field data.

7.1 The resource model as a standalone app (Simulink Compiler)

The district resource model is packaged with **Simulink Compiler** as `NetraSetuResourceModel.exe` (`simulink-model/deployable/`). It runs on any Windows PC with the free MATLAB Runtime R2026a and needs no MATLAB or Simulink licence.

What it does:

- **Inputs:** you set patients per year, PHCs, ophthalmologists, tier mix, review times, bandwidth and days.
- **Results:** KPIs (utilisation, mean and p95 review wait, end-to-end time), the bottleneck with a plain-language recommendation, and queue and utilisation curves over time.
- **Minimum ophthalmologists:** it searches for the smallest team that holds the p95 review wait under 60 minutes, for routine and camp-mode schedules.
- **Headless mode:** `--json` takes parameters in and writes results out, for use from the backend.

Why it runs a different model. Simulink Compiler deploys through Rapid Accelerator, which needs C code from every block, and SimEvents blocks do not support code generation (`SimulinkEventEngine:Engine:CodeGenNotSupported` , verified 2026-10-03). So the app simulates `districtResourceModel.slx` : a clock-driven, code-generation-capable MATLAB System block (`DistrictScreeningEngine.m`) that implements `referenceQueueingModel.m` 's algorithm, including Tier C pre-empting Tier B with resume. It draws its random inputs in the reference model's order, so for the same parameters and seed it reproduces the reference **exactly**. Every parameter is tunable at runtime, with no rebuild.

Verification (`validateDeployableResourceModel`):

Mode	Result
Normal simulation	5/5 scenarios exact ($\leq 1e-14$ relative)
Rapid Accelerator deployment mode	5/5 scenarios exact
The compiled exe	5/5 scenarios exact, same bottleneck verdicts

The SimEvents `.slx` remains the PS deliverable and the weekly validation model.

7.2 The interactive pipeline dashboard as a standalone app (Simulink Compiler)

`netraSetuPipeline.slx` is the SimEvents model of the whole pipeline, run as a live dashboard in Simulink. It is also packaged as `NetraSetuPipelineDashboard.exe` (`simulink-model/deployable/pipeline/`).

Stages: arrivals, capture with retakes, PHC sync queue, network upload, grading with failure and retry, tier triage, reviewers with pre-emption, referral.

Live controls, changeable while the simulation runs: patients per hour, review speed, network link on/off, grading on/off.

Live displays: backlogs, outcome counters, load gauges, reviewer lamps, and queue curves over time.

How it is live in a compiled app. The controls are root inports of `netraSetuPipelineLive.slx`, fed each step through `simulink.compiler.setExternalInputsFcn`. The displays read the model's output through `setExternalOutputsFcn`, and `setPostStepFcn` paces the run.

How the model works. The engine (`PipelineEngine.m`, a code-generation-capable MATLAB System block) processes every event at its exact continuous time. One deliberate improvement over the `SimEvents` model: an outage stops new work from starting, instead of being modelled as an ~11-day service time that strands the case in service.

Verification (`validateLivePipelineModel`):

- the model is exact against the engine, in normal and deployment mode;
- a live network outage builds the PHC backlog $0 \rightarrow 96 \rightarrow 0$, and a live grading outage builds the grading backlog $0 \rightarrow 91 \rightarrow 0$ with the reviewers idle;
- a 300 h run is within 1.2% of queueing theory on every check;
- the compiled exe matches the deployment-mode run exactly.

Findings about the `SimEvents` dashboard model (left unchanged):

- its grading-retry loop **deadlocks**: a failed case re-enters the same full capacity-2 server, and grading stops at 13.4 h in the default 16 h run, stranding 128 cases;
- it generates ~10% more patients than its arrival rate, because of its once-a-second sampled random signal.

8. Resilience and edge-case handling

- **Extended outages:** urgency-then-age sync ordering and PHC last-contact time feeding System Health apply to both front-ends. The mobile app additionally sends a lightweight case-summary packet ahead of the full image on thin connectivity, and shows a storage-pressure warning as its local queue fills; the desktop app posts the full image directly, with chunked upload for large files. A manual export-to-drive fallback for multi-day outages is a planned addition, not yet built on either front-end.
- **Ungradable images:** after a fixed number of failed retakes, the technician can mark a capture "best effort — ungradable." The flag is sent to and stored by central, but central does not yet read it to force Tier C — that routing effect is a stated follow-up, not current behavior.
- **Duplicate patients:** fuzzy matching at registration; the technician confirms or merges.
- **Per-eye capture:** one visit can produce two independently graded cases.
- **Failed patient contact:** an undeliverable SMS flips the referral to manual follow-up.
- **Collision-safe identity:** PHC code + timestamp + random suffix, with `capture_id` as an idempotency key enforced by a uniqueness constraint.
- **Active monitoring:** silent PHC, stuck job, dead MATLAB session and stale referable case are treated as one failure class, surfaced on one screen.
- **MATLAB session supervision:** the session auto-restarts, and health is reported per component (database, queue, MATLAB, Python, segmentation worker). A case that fails while MATLAB is down is currently marked failed and must be resubmitted; automatic re-queue on recovery is not implemented.
- **Concurrent review:** opening a case claims it.
- **Disagreement:** Confirm is never available when the branches disagree.

9. Security (current state)

What's actually enforced today:

- **Central:** authentication is enabled.
 - Passwords are hashed with `bcryptjs`; a wrong password gets a genuine 401.
 - The session is a signed JWT (HS256) in an **httpOnly** cookie, 12 h by default, `Secure` (`services/authConfig.js` , `services/authTokens.js`).
 - Role-based access control is enforced server-side on every endpoint, including media: 401 without a session, 403 for the wrong role. The UI routes enforce the same roles.
- **PHC ingestion:** a per-PHC API key is required on every central ingestion call. Only its hash is stored. Keys are provisioned via CLI, shown once, and never returned by any API.
- **PHC desktop:** technician accounts hashed with `scrypt` (`phc-local-app/backend/services/passwords.js`), provisioned via CLI. Login is on by default, because the PHC backend listens on the clinic LAN for phone pairing. The mobile app authenticates against the paired PHC PC over a sealed peer channel.
- **Encryption at rest (central):** AES-256-GCM for stored images, Grad-CAM overlays and report PDFs (`MEDIA_ENCRYPTION_KEY`).
- **Encryption in transit:** TLS is supported by both backends, and provided by the reverse proxy in deployment (§12).
- **Audit logging:** every access to patient data is recorded in `access_log` .

Stated gaps. This is a prototype-stage security floor, not a production compliance claim:

- The PHC-side local stores (desktop SQLite and mobile database/photos) are **not encrypted** this round.
- There are no refresh tokens, no rate limiting and no formal secret rotation.
- No penetration test or security audit has been performed.

10. Honest limitations (stated on purpose)

1. **Decision support, not diagnosis.** This is a screening decision-support system, not an autonomous diagnostic. Every positive is confirmed by an ophthalmologist.
2. **Domain shift.** Sensitivity is 95.0% in-domain but 75.2% on an unseen camera at the shipped threshold. Local recalibration does not close this gap, because ranking quality is the limit, so unvalidated cameras are always routed to human review.
3. **Top-grade resolution.** Exact grade-4 recall is 57.4% in-domain. It is made safe by threshold-based referral and CNN-grade-4 → Tier C routing, not by the recall itself.
4. **MATLAB licensing, mostly solved.** The PHC quality gate, the central rule engine, and the classifier and segmentation nets all now run as compiled standalone executables on the free MATLAB Runtime, no license needed on the serving machine. By default, central inference still runs on a licensed MATLAB session in day-to-day operation; the compiled path exists and is verified, but switching the default over is the remaining step. See §12 for every standalone application and its status.
5. **Early red-lesion evidence.** The deployed microaneurysm/haemorrhage model was evaluated on 16 images, the same ones it was tuned on.
6. **Neovascularization and uncovered lesion types.** Neovascularization failed validation and is not used. Cotton-wool spots, venous beading and IRMA are not detected.
7. **Rule engine accuracy.** Exact agreement is 60.2% (n = 103).
8. **Vessel threshold.** There is a cross-dataset gap, and the fixed 0.5 threshold is not domain-adaptive.
9. **In-memory grading queue.** Failed-while-MATLAB-down cases are not auto-re-queued.
10. **Partial test coverage.**

- conformal: 90/90;
- fovea gate: 12/12;
- PHC backend: 41;
- mobile: 28;
- central backend: no automated suite.

11. **Simulink parameters** are modeled assumptions, not field data.
12. **DME.** Diabetic macular edema cannot be reliably ruled out from colour fundus photographs alone.
13. **No clinical data.** No real-patient or prospective clinical data has been used.
14. **No subgroup analysis.** No subgroup (demographic) analysis has been done: the public datasets don't carry the needed fields.
15. **Quality gate unvalidated against labels.** The quality gate's thresholds are engineered heuristics, not validated against a gradability-labelled dataset.

11. Datasets

Dataset	Size	Used for
APTOS 2019 (Aravind Eye Hospital, India)	3,662 images, 5-class grade	Classifier training and held-out test
IDRiD (Nanded, Maharashtra, India)	516 grading · 81 segmentation · 516 localization	Classifier training and test, lesion and localization models, rule-engine calibration, official test-split evaluation
EyePACS	Curated subset (10k–24k of 35,126, quality-filtered)	Deployed classifier training (5-class loss, weight 0.5)
CHASE_DB1	28 images	Vessel segmentation training / held-out evaluation
DRIVE	20 test images	Vessel cross-dataset evaluation only
Messidor-2	1,744 gradable images, 874 patients	External validation only; never used for training, calibration or thresholds

Only public research datasets are used anywhere in this system, and no real patient data. Two of the core training datasets, APTOS and IDRiD, come from Indian populations and clinics, which is directly relevant to the deployment target. The external test, Messidor-2 (France), deliberately comes from a different population and camera.

12. Deployment

12.1 Target production deployment (the plan)

Component	Where it runs	How
PHC desktop app + PHC local backend	On-site at each PHC, on the technician's PC	Installed locally, fully offline-capable. SQLite queue, MATLAB-compiled quality gate on the free MATLAB Runtime (no license needed)
Mobile app	Technician's Android phone	Installed APK. On-device JS quality gate, expo-sqlite queue

Central backend + grading pipeline	A district/state-hosted Linux server or VM	Docker Compose behind a TLS reverse proxy. MATLAB inference packaged with MATLAB Compiler and run on the free MATLAB Runtime, removing the license dependency. Python worker for preprocessing and segmentation. PostgreSQL with nightly backups
Central web (ophthalmologist / admin)	Static hosting	Same-origin with the API (reverse proxy or host rewrites), so the session cookie and image access work without cross-site exceptions

Why this shape. Grading needs the full-capacity models and a single calibrated environment, so it stays central. Offline capture and quality gating are what a rural PHC actually needs locally.

12.2 Current deployment (online submission demo)

Component	Hosted on	Link
Central web (ophthalmologist / admin)	Vercel	https://centralsys.vercel.app
PHC desktop web (hosted demo station)	Vercel	https://phcapp.vercel.app
Central backend (API)	Render (Docker, free tier)	https://netrasetu-central.onrender.com/health
PHC local backend (API)	Render (Docker, free tier)	https://netrasetu-phc.onrender.com/health
ML inference service	Render (Docker, free tier)	internal, reached by the central backend over HTTP
Database	Render Postgres (free tier)	internal
Mobile app	EAS-built standalone APK, direct install	https://expo.dev/artifacts/eas/AXeqZ3jshQkh7qTIsaXkO3nUBWXygH9XnyRv1

What a judge can do on the demo. Log in to the PHC web app with the demo technician account (demo / Fundus-Comet-52), register a patient, capture or upload a fundus image, and watch it move through the quality gate, sync to central, and reach graded status end to end — this exact flow has been run live against the hosted deployment, not just locally. On the central web app, log in as the demo ophthalmologist or district admin to review the queue, open a case's full rationale (Grad-CAM, lesion evidence, both branches' grades), confirm or override, and see the referral/admin views. The Android app installs the same way — download and open the APK directly, log in

with the same demo technician account, and the whole capture flow works against the hosted backend with no pairing step and no developer machine required.

Why this deployment exists, and how it differs from §12.1's target architecture. §12.1 is the system as designed for a real district rollout: MATLAB-native throughout, on infrastructure with a MATLAB license. This section's deployment answers a different, narrower question — *"can a judge reach a fully working system from anywhere, with no local setup, around the clock?"* — and it is built specifically to survive on infrastructure that has **no MATLAB at all**:

- **Classifier and segmentation** run on ONNX Runtime, the same model weights, numerically verified equivalent to the MATLAB path (§6.7, §6.5, §6.6) — this is not a lower-fidelity stand-in, it is the identical trained network served a different way.
- **The quality gate and rule engine** run on their verified JS ports, the same ports described in §4.2 and §6.7 — both checked against their MATLAB originals with zero mismatches across hundreds of test cases (ML_BENCHMARKS.md §7).
- **Every case still records which engine actually produced each result** — this deployment never claims an engine ran when it didn't; it is honest about its own configuration in its own output, the same standing rule that governs every other part of this system (§1).

What's different in practice, honestly:

- Render's free tier spins services down after ~15 minutes idle; the first request after idle is slower while it wakes back up.
- Media storage on this deployment is ephemeral (resets on redeploy) — the local/offline deployment does not have this limitation, since it owns its own disk.
- SMS referral notifications are wired for real sending (not a dry-run), but a Twilio trial account only delivers to manually-verified numbers — a judge's phone may not receive the SMS step live unless pre-verified, even though the referral itself completes correctly.

Standalone MATLAB applications, for the local/offline architecture. Alongside the hosted demo, several components are also packaged as standalone executables — MATLAB Compiler or Simulink Compiler plus the free MATLAB Runtime, no license needed to run them — specifically to prove out §12.1's licensing-free target architecture:

Application	What it packages	Status
PHC quality gate	Focus/FOV/illumination/glare checks	Built
District resource-allocation model (CLI)	The staffing-recommendation queueing model	Built
District resource-allocation model (interactive app, §7.1)	The same model, with KPIs and charts	Built
Central case-chain engine	Rule engine, camera check, evidence text	Built
Central inference engine	Classifier + all four segmentation models	Built
Clinical-rationale PDF report generator	The per-case evidence PDF	Built
Interactive full-pipeline dashboard (§7.2)	The watchable whole-pipeline demo, live controls mid-run	Built

These are build artifacts, not part of the hosted demo's request path; they exist to demonstrate that the licensing-free deployment story in §12.1 is real and buildable, not aspirational.

Not packaged as a standalone app, by decision: the weekly SimEvents self-validation (§7). It's a scheduled background comparison the central backend already runs on its own, with no interactive use a standalone app would

add — the same underlying model is already the one judges can run and watch, as the interactive dashboard above.

12.3 Standalone offline PHC station, and other downloadable components

A third, practical deployment: a self-contained downloadable package (the compiled quality gate plus the free MATLAB Runtime, no separate MATLAB install needed) that turns any Windows PC into a working PHC capture station in minutes. It registers as its own site — so its cases never mix with the hosted demo PHC's — and syncs to the same central backend as the rest of the system. Tested end to end: registration, capture, the compiled quality gate, sync, central grading, and the result returning to the station, including restarts and working offline.

Component	Where	Needs
Standalone PHC station	Zip, distributed outside git	Windows + the free MATLAB Runtime; its own site credentials issued by central
District resource-model app (CLI)	In the repo	The free MATLAB Runtime
District resource-model app (interactive, §7.1)	In the repo	Windows + the free MATLAB Runtime with the Simulink Compiler add-on
Interactive pipeline dashboard app (§7.2)	In the repo	Windows + the free MATLAB Runtime with the Simulink Compiler add-on
Central case-chain engine	In the repo	The free MATLAB Runtime
Clinical-rationale PDF report generator	In the repo	The free MATLAB Runtime
Central inference engine (classifier + segmentation)	GitHub Release — too large (265 MB) for a normal git push	The free MATLAB Runtime
Compiled quality gate (also bundled in the standalone PHC)	In the repo	The free MATLAB Runtime